

Projektuppgift

Frontend-baserad webbutveckling, DT211G

Recipe Roulette

Slumpade recept från API-data

Simon Nilsson



Mittuniversitetet

MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Simon Nilsson, sini2500@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: VT, 2026

Sammanfattning

Den här rapporten gäller utvecklingen av en webbapplikation kallad Recipe Roulette. Det är en API-fokuserad webbapp som kombinerar matrecept, geografisk data och fakta om länder för att ge användare en rolig upplevelse som både ger inspiration för matlagning och lite geografisk kuriositet.

Målgruppen för webbapplikationen är matintresserade användare som är intresserade att prova på recept från olika länder och kulturer. Genom att klicka på en "slumpa recept"-knapp kan användare få ett nytt recept, information om landet receptet kommer från, samt en karta med landet markerat.

Projektet gick ut på att skapa en "mashup-applikation" som hämtar data från flera JSON-baserade webbtjänster via javascript fetch. Inga frontend-ramverk användes under arbetet. Följande API:er har använts; TheMealDB [1] för recept, REST Countries [2] för data om länder och Nominatim [3] i kombination med OpenStreetMap [4] för att visa kartor inuti en iframe. Eftersom dessa API:er levererar data på engelska gjordes applikationens gränssnitt också på engelska.

Webbapplikationen har utvecklats med hjälp av HTML, JavaScript och SCSS som del av en automatiserad arbetsprocess med Node.js och Vite. Versionshantering gjordes på Github. Responsiv design användes för att få appen att fungera bra på både stora och små skärmar. CSS-animation användes för att ge användare en mer interaktiv upplevelse när de slumpar fram nya recept.

Detta projekt visar att det är möjligt att kombinera flera API:er till en sammanhängande och användarvänlig webbapplikation.

Innehållsförteckning

Sammanfattning.....	iii
1 Introduktion.....	v
2 Teori.....	vi
2.1 API och JSON.....	vi
2.2 Mashup-applikationer.....	vi
2.3 Fetch API.....	vi
2.4 SCSS och CSS.....	vi
2.5 Responsiv webbdesign.....	vii
2.6 API i projektet.....	vii
2.7 Versionshantering.....	vii
2.8 Automatiserad arbetsprocess.....	vii
3 Metod.....	viii
3.1 Utvecklingsmiljö.....	viii
3.2 API-hantering.....	viii
3.3 Rendering av information.....	ix
3.4 Responsiv design och SCSS.....	ix
3.5 Animeringar och interaktion.....	ix
3.6 Felhantering.....	ix
4 Konstruktion.....	x
4.1 Projektstruktur.....	x
4.2 HTML och layout.....	x
4.3 API-integration.....	xi
4.4 Dynamisk rendering.....	xi
4.5 Felhantering.....	xii
4.6 Responsiv design.....	xii
4.7 SCSS och design.....	xii
4.8 Animationer.....	xii
4.9 Optimering av bilder.....	xii
5 Resultat.....	xiii
6 Slutsatser.....	xiv
Källförteckning.....	xv
Bilaga A: Wireframe.....	xvi
Bilaga B: Grafisk profil.....	xvii

1 Introduktion

Det finns otaliga webbtjänster som tillhandahåller någon typ av hjälpsam data via ett API. Genom att använda och kombinera dessa går det att skapa så kallade ”mashup-applikationer” där material från flera källor smälter samman till en ännu bättre webbtjänst. Det här projektuppgiften gick ut på att skapa en sådan applikation med hjälp av enkla verktyg, som JavaScript fetch, och utan att behöva förlita sig på något stort frontend-ramverk.

Målet med det här projektet var att skapa en webbapplikation som kombinerar matrecept, geografisk data och fakta om länder till en enhetlig användarupplevelse. Resultatet är Recipe Roulette, en webbapplikation där användaren kan slumpa fram recept från olika länder och få information om rättens ursprungsland, samt en karta med landet markerat. Tanken är att applikationen ska ge inspiration för matlagning och väcka intresse för olika platser runt om i världen.

Målgruppen är matintresserade användare som vill prova på recept från olika delar av världen och kanske till och med lära sig något nytt om dem, eller i alla fall kunna hitta landet på kartan.

Projektet var del av kursen Frontend-baserad webbutveckling och flera tekniska krav ställdes på arbetet. Webbapplikationen skulle använda minst två API:er, använda responsiv design, SCSS och innehålla CSS-animationer. En automatiserad arbetsprocess med Node.js och Github skulle också användas. För att hålla API:erna i fokus skulle även majoriteten av applikationens material komma från API:er och uppdateras dynamiskt via JavaScript.

2 Teori

Här finns förklaringar av de tekniker och begrepp som använts för utvecklingen av Recipe Roulette. Störst fokus ligger på hantering och presentation av data från API-tjänster.

2.1 API och JSON

Ett API (Application Programming Interface) möjliggör kommunikation mellan olika system genom att exponera specifika ändpunkter för olika syften. För webbapplikationer används API:er ofta för att låta andra hämta data på ett strukturerat sätt. Data skickas vanligtvis i JSON-format, ett textbaserat dataformat som är lätt att bearbeta med JavaScript.

2.2 Mashup-applikationer

Att kombinera data från flera olika API-källor kallas ofta en ”mashup”. Målet är vanligtvis att skapa en tjänst som ger mer värde än varje API kan ge på egen hand. Utvecklaren av en mashup-applikation kan välja specifika delar av det API-utvecklare erbjuder för att skapa sin önskade användarupplevelse.

2.3 Fetch API

JavaScript erbjuder ett verktyg kallat ”Fetch API” [5] som används för att göra asynkrona förfrågningar till externa webbtjänster. En asynkron förfrågan till ett API görs i bakgrunden, utan att hindra användaren från att interagera med resten av webbapplikationen.

2.4 SCSS och CSS

SCSS är en utökning av vanlig CSS med extra funktioner och praktiska konventioner som förenklar arbetet med stora projekt. Det används genom Sass, en CSS-preprocessor, som lägger till ett steg i arbetsprocessen där SCSS kompileras till CSS och optimeras för leverans i en så liten fil som möjligt. SCSS gör det möjligt att arbeta på ett effektivt sätt och ändå få ren CSS i slutändan.

2.5 Responsiv webbdesign

Responsiv webbdesign betyder att en webbplats eller applikation tar hänsyn till användarens skärmstorlek. Det kan hända att layout justeras justeras, element döljs eller modifieras, eller ordningen av information ändras. CSS används för att göra detta möjligt, via t.ex. Flexbox, CSS Grid och media queries. Målet är att skapa en användarupplevelse som fungerar på mobil, surfplatta och dator.

2.6 API i projektet

Recipe Roulette använder sig av fyra separata API:er för att skapa önskad funktionalitet.

- TheMealDB används för att hämta slumpmässiga recept.
- REST Countries används för att hämta landsinformation.
- Nominatim används för att hämta geografiska koordinater.
- OpenStreetMap används för att visualisera en iframe-karta.

2.7 Versionshantering

Git är ett versionshanteringssystem som används för att spåra förändringar i filer. Det ger utvecklare möjligheten att dela upp arbetet i grenar, hitta problem, jämföra versioner av filer, m.m.

GitHub är en publiceringstjänst för versionshanterad kod som gör det lättare att visualisera projekt och dela källkod med andra.

2.8 Automatiserad arbetsprocess

Vite är ett byggverktyg som används i kombination med Node.js för att automatisera arbetet. Det hjälper till med hantering av Node-moduler, körning av CSS-preprocessorer och optimering av filer för publicering. Vite har även en lokal utvecklingsserver. JSDoc är ett verktyg som genererar dokumentation för JavaScript-projekt och kan köras i samband med att Vite bygger filerna för publicering.

3 Metod

Utvecklingen av Recipe Roulette har gjorts med en JavaScript-centrerad arbetsprocess. Användning av API:er, hantering och presentation av data har delats upp i separata funktioner.

3.1 Utvecklingsmiljö

Projektet har Node.js och Vite som grund för en automatiserad arbetsprocess. Med Vite som byggverktyg användes verktyg för optimering av kod, en utvecklingsserver och kompilering av SCSS. Git och GitHub användes för versionshantering under projektet. För dokumentering av projektet användes JSDoc i kombination med Vite, så webbsidor för dokumentation skapades automatiskt.

3.2 API-hantering

Projektet använder JavaScripts Fetch API och asynkrona funktioner (async/await) för att hämta data från API-tjänster på följande vis.

- 1) Ett slumpmässigt recept hämtas från TheMealDB. Bland receptdatan finns namnet på ett land som används i nästa API.
- 2) En förfrågan görs till REST Countries API med namnet på landet och tjänsten returnerar information som flagga, språk, befolkning och huvudstad.
- 3) Landets namn skickas även till Nominatim för att få tillbaka koordinater som kan användas för att skapa en karta.
- 4) Koordinaterna används för att uppdatera en OpenStreetMap iframe så att den visar en karta med korrekt land marketat.

Denna kedja av API-aktivitet är kärnan av webbapplikationens funktionalitet.

3.3 Rendering av information

All data från API-tjänsterna presenteras med hjälp av JavaScript på specifika platser i HTML-strukturen. Receptets instruktioner och ingredienser, landets information och karta har sina sektioner. Varje gång ett nytt recept hämtas visas ett ”laddar”-meddelande för användaren och innehållet av de olika HTML-elementen byts ut. Detta är klassisk DOM-manipulering med JavaScript.

3.4 Responsiv design och SCSS

Webbapplikationens stilar har konstruerats med SCSS och saker som variabler, mixin och nästlade stilregler har använts. Layouten består huvudsakligen av ett CSS grid med separata ”kort” för varje element. Dessa kort presenteras i antingen tre eller en kolumn baserat på användarens skärmstorlek, via CSS media queries.

3.5 Animeringar och interaktion

CSS-animering används för att förbättra användarupplevelsen på en specifik plats, när användaren klickar på ”nytt recept”-knappen. Denna knapptryckning utlöser en animering med ett snurrande roulette-hjul som finns precis bredvid knappen. På detta vis får användaren feedback om att ett nytt recept hämtas. Det visas även text på platsen receptet kommer vara för att indikera att det laddar.

3.6 Felhantering

Om något går fel under API-anrop måste det visas för användaren. JavaScripts try/catch-block används för detta. När ett fel uppstår visas felmeddelandet där receptet skulle hamnat.

4 Konstruktion

Den här sektionen beskriver implementeringen av Recipe Roulette.

4.1 Projektstruktur

Projektet utvecklades med en så enkel filstruktur som möjligt. Ett Node-projekt skapades med Vite, Sass och JSDoc som moduler. Endast en html-fil användes för appen och en enda javascript-fil användes för all funktionalitet, men flera filer skapades för att organisera SCSS på ett effektivt sätt. SCSS kompilerades till en enda CSS-fil vid byggsteget så slutresultatet blev så här:

```
dist/
├── assets/
│   ├── main.css
│   └── main.js
├── docs/
├── img/
│   ├── wheel.png
│   └── logo.png
└── index.html
```

4.2 HTML och layout

Webbapplikationen byggdes med responsiv layout baserad på CSS Grid och Flexbox. För större skärmar användes 3 kolumner, och på mindre skärmar endast 1 kolumn. Information om receptet presenteras i de två första kolumnerna, med hjälpsam separation mellan bild, namn, ingredienser och instruktioner. Den tredje kolumnen har en bild på ett roulette-hjul och en knapp för att slumpa fram ett nytt recept, samt information och karta för landet det aktuella receptet kommer ifrån.

HTML-strukturen består på hög nivå av tre semantiska element: header, main och footer. Den huvudsakliga layouten finns i main-elementet och det finns flera underliggande div-element för de olika ”korten”. Tydliga klassnamn används för CSS-styling och ID för enkel identifiering med JavaScripts QuerySelector.

4.3 API-integration

När sidan laddas för första gången och när användaren klickar på ”slumpa recept”-knappen startas en asynkron funktion som sammanställer all information och sedan renderar den i HTML.

API-anropen använder Fetch API och har sina egna funktioner.

```
const recipe = await getRandomRecipe();  
let country = recipe.strCountry;  
const countryInfo = await getCountryInfo(country);  
const coords = await getCoordinates(country);
```

Den första funktionen ”getRandomRecipe” anropar TheMealDB på ändpunkten </api/json/v1/1/random.php> vilket ger ett slumpmässigt recept. Receptets ursprungsland sparas i variabeln ”country”.

Den andra funktionen ”getCountryInfo” hämtar information om landet från REST Countries, och ”getCoordinates” hämtar koordinater från Nominatim för användning i kartvisningen.

4.4 Dynamisk rendering

Data som hämtats från API-tjänster presenteras för användaren via tre olika funktioner för de relevanta delarna av HTML-sidan.

```
renderRecipe(recipe);  
renderTrivia(countryInfo);  
updateMap(coords.lat, coords.lon, country);
```

Dessa funktioner använder sig av .innerHTML för att byta ut material, och JavaScript template literals används också flitigt.

```
triviaContainer.innerHTML = `  
<div class="card">  
  
<h2>${country.name.common}</h2>
```

Viss data bearbetas med loopar eller joins för att få dem till ett mer användbart format än API-tjänsten tillhandahöll. Detta gjordes för recepts ingredienser som levererades separat från måttenheter och recepts instruktioner som levererades i en enda lång sträng.

4.5 Felhantering

Eftersom applikationen använder data från tredjepartstjänster kan saker gå fel på vägen. Därför implementerades try/catch-block och "throw Error" för felmeddelanden. Här är ett exempel:

```
async function getCountryInfo(country) {  
  const res = await fetch(`https://restcountries.com/v3.1/name/${country}?  
    fullText=true`);  
  if (!res.ok) throw new Error("Failed to get country info, sorry!");  
  const data = await res.json();  
  return data[0]; }  
}
```

4.6 Responsiv design

Layout justeras med hjälp av media queries för att ge en bättre upplevelse på små skärmar, där en helt vertikal layout är enklare.

En hamburgermeny har implementerats för att dölja menylänkar, även om webbapplikationen bara har en länk och en dark-mode toggle i menyn. Denna meny demonstrerar hur innehåll kan döljas och visas genom att lägga till och ta bort css-klasser från element.

4.7 SCSS och design

SCSS användas för att organisera stilar för css-reset, layout, bas-element och komponenter. Variabler skapades för färger och skuggor, och en SCSS-mixin användes för mobillägets media query.

SCSS skapades för en mörk och ljus färgprofil som användaren kan växla mellan genom en knapp i menyn. Detta fungerar genom att lägga till eller ta bort en CSS-klass på body-elementet.

4.8 Animationer

En CSS-animation implementerades med keyframes för att rotera ett roulette-hjul när en viss JavaScript-funktion körs via en knapp.

4.9 Optimering av bilder

De få bilder som finns i webbapplikationen är optimerade för mindre storlek och i det moderna formatet webp. PNG-alternativ finns också för situationer som inte stödjer webp.

5 Resultat

Det här projektet resulterade i en responsiv mashup-applikation som kombinerar data från flera API-källor till en enhetlig användarupplevelse. Applikationen presenterar receptdata, geografisk information om receptets ursprungsland och en karta. Allt detta görs utan att ladda om sidan, tack vare asynkron Fetch.

När sidan laddas eller knappen för nytt recept klickas så utlöses en rad API-anrop. Ett recept hämtas från TheMealDB, inklusive namn på det land som maträtten kommer ifrån. Detta land används för att hämta information från REST countries API, och en karta visas med hjälp av Nominatim och OpenStreetMap.

Användare kan enkelt se följande information:

- En bild på maträtten
- Maträttens namn och ursprungsland
- En länk till receptets källa på nätet
- Receptets ingredienser och instruktioner
- Landets flagga, huvudstad, region, befolkning och språk

Applikationen har en responsiv design som fungerar lika bra på både stora och små skärmar utan att förlora någon information.

När användaren trycker på knappen för att få ett nytt recept spelas en snurr-animation på en bild av ett roulette-hjul, vilket ger visuell feedback om vad som händer. Samtidigt visas även "laddar"-text på platserna där information senare kommer fyllas in.

6 Slutsatser

I det här projektet kan vi se att det är möjligt att göra en mashup-applikation som kombinerar data från flera API-tjänster med väldigt få verktyg. Inga stora ramverk eller bibliotek behövdes.

Att använda resultat från ett API-anrop som inmatning till nästa anrop gjordes enkelt med asynkrona funktioner. Genom att hämta data och välja ut relevanta delar för användningsfallet gick det att skapa en mer unik och intressant webbapplikation.

Mindre problem uppstod vid hämtning av data på grund av hur API-leverantören valt att dela upp ingredienser och måttenheter, men detta löstes med lite extra bearbetning i JavaScript. Vissa dataobjekt saknade även detaljer, vilket leddet till behovet att hantera tomma rader och null-värden. Detta visar att man inte alltid kan få perfekt data från API:er, och att validering alltid krävs.

Genom att animera endast en del av webbapplikationen, roulette-hjulet, blev det tydligt hur mycket skillnad ett enda element kan göra på användarupplevelsen. Ett snurrande hjul är en mycket bättre visualisering av väntetid än en knapp som inte gör något.

Projektet uppnådde målen för uppgiften som fokuserade på API-användning, men skulle med fördel kunna vidareutvecklats med snyggare design, mer interaktiv karta, receptsökning eller filtrering, fler sätt att få fram recept, o.s.v.

Arbetet gav praktisk erfarenhet av API-tjänster, asynkrona funktioner i JavaScript och arbetsprocessen med Node och Vite.

Källförteckning

- [1] TheMealDB
<https://thymealdb.com/>
Hämtad 2026-06-06

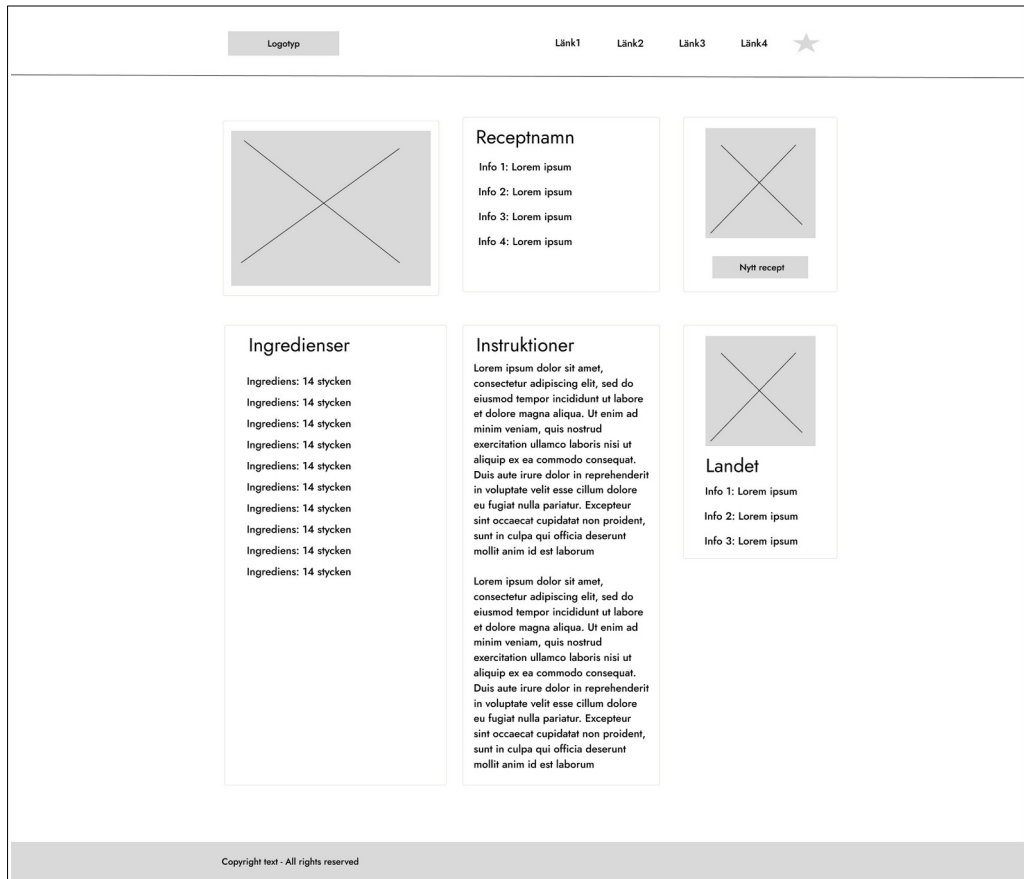
- [2] REST Countries
<https://restcountries.com/>
Hämtad 2026-06-06

- [3] Nominatim
<https://nominatim.org/>
Hämtad 2026-06-06

- [4] OpenStreetMap
<https://www.openstreetmap.org/>
Hämtad 2026-06-06

- [5] Mozilla, Fetch API
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
Hämtad 2026-06-06

Bilaga A: Wireframe



Bilaga B: Grafisk profil

